



Sahar Saleh - M00998448

Tanzira Khan Khanam- M00996376

Louisa Karpa – M00949803

Kai Wallace-Sahel

Mohamed Amine Bennacer – M01010322

KSLAT– NATURAL GALLERY MUSEUM WEBSITE REPORT

Software Engineering Management and
Development Module

Introduction

The **Natural Gallery** is a group-developed museum website that allows visitors to explore what's on and complete practical tasks such as booking visits and managing membership tickets. The system was implemented as an **ASP.NET Core MVC** web application backed by **Azure SQL**, using **Entity Framework Core** to map **C# models** to relational tables and **Razor views** to render pages consistently across the platform. The scope of the project extends beyond a static brochure website, incorporating event and tour listings, detail pages, availability-aware booking workflows, user accounts with session-based login, saved favourites, and a navbar search feature that quickly returns matching events and tours.

From a user perspective, the system is designed for museum visitors who want to plan a trip by browsing, reading details, and making bookings, as well as registered members who require continuity through saved items and ticket history. From a software engineering perspective, the project also demonstrates the integration of server-side validation, database persistence, and client-side interactions, such as live search and asynchronous favourite updates, within a coherent MVC architecture.

This report is organised into five sections following the coursework structure: **Introduction** (project aims and context); **Design** (data modelling choices); **Testing** (testing strategy and representative test cases); **Conclusion** (summary, limitations, reflection, and future improvements); and **References** (Harvard-style citations).

Design

The project followed an iterative design approach rather than an upfront software design process. At the beginning, the group captured requirements as practical bullet points, such as browsing tours and events, booking visits, and accessing visitor information. These requirements were then translated into page-level structure and feature priorities.

The visual and interactive design was developed through a Figma prototype, which acted as the main blueprint for layout, navigation, typography, and page hierarchy before implementation. This approach suited the team context and deadline, as it enabled quick agreement on user-facing outcomes while allowing technical decisions to evolve during development.

A limitation of this method was that some lower-level technical rules, such as naming consistency, route consistency, and content structure across pages, were refined later during coding rather than being fully defined during the design stage.

(See the GitHub Repo README for the Figma Link)

The application uses a relational data model implemented with Azure SQL and Entity Framework Core, represented through C# model classes. This was appropriate because the

system manages structured entities with clear relationships and integrity requirements, including users, bookings, events, tours, tickets, and favourites.

Key Structures and Rationale

Event and Tour entities store core content such as titles, descriptions, image paths, dates, locations, and capacities. This supports filtering, searching, detail pages, and booking constraints.

Booking entities (*EventBooking* and *TourBooking*) store each booking transaction, including selected date/time, quantity, contact details, and generated ticket codes. This preserves auditable booking history and enables availability checks.

Ticket entity links user accounts to specific EventBooking or TourBooking rows so the membership area can show user ticket history. These separates booking transactions from member-facing ticket records, improving clarity.

Favourite entity represents a user-to-item saved relationship for events or tours. This provides reusable saved-items functionality without duplicating content tables.

Session-based user state stores login status and user ID during requests. This was a lightweight solution suitable for coursework-scale authenticated features such as favourites and viewing personal bookings.

Algorithm Analysis – Pseudocode examples

Availability-aware booking	Favourite toggle (AJAX pattern)
<pre> FUNCTION BookItem(itemId, bookingDate, bookingTime, quantity, email): item <- GET item by itemId IF item does not exist: RETURN error "Not found" VALIDATE bookingDate, bookingTime, quantity, email IF validation fails: RETURN error bookedCount <- SUM existing bookings WHERE ItemId = itemId AND BookingDate = bookingDate AND BookingTime = bookingTime spotsRemaining <- item.SpotsPerSlot - bookedCount IF quantity > spotsRemaining: RETURN error "Insufficient spots" ticketCode <- GENERATE unique code SAVE booking record IF user is logged in: SAVE linked ticket record RETURN success with booking confirmation END FUNCTION </pre>	<pre> FUNCTION ToggleFavourite(userId, itemId, itemType): IF userId is null: RETURN { success: false, message: "Login required" } existing <- FIND favourite by (userId, itemId, itemType) IF existing exists: DELETE existing state <- false ELSE: INSERT favourite(userId, itemId, itemType) state <- true RETURN { success: true, favoured: state } END FUNCTION </pre>

Testing

Testing was carried out as a scenario-based manual testing process aligned to end-user journeys, rather than automated unit/integration test suites. The team focused on validating critical workflows across navigation, booking, account-related actions, and data persistence. Tests were executed incrementally during development and again after major changes to confirm that updates had not broken previously working behaviour.

The approach combined:

- functional testing (does each feature work as intended),
- validation testing (do input rules and error messages behave correctly),
- state/permission testing (different outcomes for logged-in vs logged-out users),
- basic regression testing (rechecking key paths after fixes).

Because the application relies on database-backed operations, expected outcomes were verified both in UI behaviour (messages, redirects, visible records) and in persisted results (e.g., bookings/tickets/favourites represented consistently in the application state).

A limitation of this testing strategy is that it depends on human execution and therefore does not provide the repeatability and broad coverage of automated test pipelines. However, for the scale and timeframe of this coursework, it provided practical confidence on the most important user-facing behaviour.

Test ID	Feature	Test Scenario	Input/Predictions	Expected Result	Actual Result	Status
TC-01	Navigation	Open main pages from Nav bar	User on home page	Each link routes to correct page without error	As expected	Pass
TC-02	Search	Search with fewer than 2 characters	Enter 1-character query in navbar search	No API result list shown	As expected	Pass
TC-03	Search	Search valid event keyword	Query matches existing event title/genre/location	Matching event appears in dropdown and opens detail page	As expected	Pass
TC-04	Search	Search valid tour keyword	Query matches existing tour title/location/duration	Matching tour appears in dropdown and opens detail page	As expected	Pass
TC-05	Search	Search unknown term	Query has no matching records	"No results found" message shown	As expected	Pass

TC-06	Event Filters	Filter events by genre	Events page loaded, select genre	Only matching event cards remain visible	As expected	Pass
TC-07	Event Sort	Sort events by A-Z / date	Events page loaded, choose each Sort option	Card order updates correctly per selected rule	As expected	Pass
TC-08	Event Booking validation	Submit booking with missing date/time	Event booking form with required fields missing	Error shown, booking not saved	As expected	Pass
TC-09	Event Booking capacity	Try to book quantity greater than remaining spots	Existing bookings reduce availability	User sees "spots remaining" error, booking blocked	As expected	Pass
TC-10	Event Booking success	Submit valid booking	Valid date/time/quantity/email	Booking saved, confirmation ticket shown	As expected	Pass
TC-11	Tour Booking validation	Submit booking with missing date/time	Tour booking form with required fields missing	Error shown, booking not saved	As expected	Pass
TC-12	Tour Booking success	Submit valid booking	Valid date/time/quantity/email/payment	Booking saved, confirmation ticket shown	As expected	Pass
TC-13	Sign Up	Sign up a new user	Create a new unique user	Account created, User can login	As expected	Pass
TC-14	Login	Login to user account	Login to a preexisting user account	User logs in	As expected	Pass
TC-15	Favourites (authenticated)	Toggle favourites on/off	Logged-in user clicks heart twice	First click saves, second removes	As expected	Pass
TC-16	Membership Tickets	View tickets after booking while logged in	Logged-in user completes booking	Tickets appear in membership ticket list	As expected	Pass

TC-17	Contact success	Fill Contact forum	Valid name/email/department/subject/message	Confirmation message shown	As expected	Pass
TC-18	Donation success	Fill Donations forum	Valid amount and billing	Confirmation message shown	As expected	Pass

Conclusion

The team worked together to develop a functional ASP.NET Core MVC website, meeting our main goal of creating a booking platform despite some challenges. The final system includes tour and event pages, detail pages, support pages (donations, privacy policy, contacts), sign up and login, booking workflows, account-based favourites and tickets, and a shared nav bar. Next, each member will share their individual reflections.

Reflections

Sahar Saleh – M00998448

As team leader for Natural Gallery, I coordinated the team, defined requirements and kept the project organised throughout development. My primary technical contribution was the “what’s on” section, where I developed and maintained key Razor views including Booking.cshtml, Index.cshtml, Ticket.cshtml, Tourdetail.cshtml and Tours.cshtml. On the backend, I worked extensively on the WhatsOnController.cs and designed and updated models such as Tour.cs, Tourbooking.cs, TicketViewModel.cs, Ticket.cs and Favourite.cs. I ensured smooth integration with MembershipController and AccountController to connect tours to membership features. Additionally, I created and refined the styling in whatson.css, separated and linked the signup and login pages for better usability, contributed to the ReadMe.md for clear project documentation and produced the video demonstration showcasing our website.

On the database side, I updated the MuseumDbContext and DatabaseSetup.sql for our Azure SQL database, ensuring all tour-related relationships were handled correctly. However, one recurring limitation I faced was with viewing saved items and tickets in the membership section. There were persistent issues where the saved favourites and tickets would not show tours correctly for logged-in users because these features were not always properly linked to the SQL database. Resolving these problems involved troubleshooting both the data models and the backend logic to achieve consistent data retrieval and display within the membership pages. Another key limitation I encountered was integrating my tours work with the events module, which was developed by another team member. Combining our separately built features required considerable troubleshooting to align data structures, controller logic and user interface elements. Additionally, technical difficulties with linking other frontend views to Azure SQL required ongoing attention. At the group level, we also had issues merging branches, committing updates and syncing Visual Studio with GitHub challenges that sometimes slowed progress and forced us to adjust our workflow.

Moving forward, I will prioritise earlier and more frequent collaboration on shared features, regular integration and joint testing as well as further team training on version control tools to improve efficiency and reduce development conflicts.

Louisa Karpa – M00949803

As the developer I built a large slice of the full stack: I implemented the core domain models (Event.cs, EventBooking.cs, Contact.cs, Favourite.cs, Ticket.cs) and supporting view models (SavedItemViewModel, TicketViewModel), built the key MVC controllers (WhatsOnController, ContactController, MembershipController) and the associated Razor views (Events.cshtml, EventDetail.cshtml, EventBook.cshtml, EventTicket.cshtml, Contact.cshtml, ContactSuccess.cshtml, Favourites.cshtml, Tickets.cshtml, Membership/Index.cshtml) and I produced the database artefacts for my tables (Events, EventBooking, Contacts, Favourites, Tickets) including a full DatabaseSetup.sql script covering the team schema, plus EF Core wiring in MuseumDbContext and initial seeding in DbSeeder. This involved Entity Framework relationships and querying patterns, server-side validation and anti-forgery protection on POST actions, session-based user state for logged-in features, JSON endpoints used by the UI (search/availability/favourites), and client-side behaviour such as filtering/sorting on the events listing. I also handled documentation (ReadMe.md) and the wider design/deployment work, such as creating the full Figma design & building the SQL database in Microsoft Azure.

A key limitation I experienced was that the WhatsOnController became the main integration hub for multiple features that were originally planned separately. In practice, the same controller ended up coordinating event and tour listings, detail pages, booking POST actions, JSON APIs for live search and availability checks, and AJAX favourites endpoints. While this approach was efficient for a group deadline because it kept related routes under one namespace, it increased coupling: small changes in one area could unintentionally affect another, and the file became difficult to navigate during code review and debugging. Reflecting on this, I would treat “controller size” as an early warning sign in future projects and split responsibilities earlier (for example, separate controllers or shared service classes) once a second domain starts reusing the same patterns. Another limitation was validation being implemented in more than one place. This duplication improves the user experience (immediate feedback) but creates maintenance risk: if one side is updated and the other is not, the application can behave inconsistently or appear to “work” in the browser while still failing server-side (or vice versa). It also encourages informal validation rules (such as simplified email checks) rather than a single authoritative validation model. In future work I would aim to centralise rules using a consistent server-side validation approach and treat client-side validation strictly as an optional enhancement layered on top, not a second source of truth.

Mohamed Amine Bennacer – M01010322

As the QA and testing developer, I implemented comprehensive unit testing using MSTest framework across all core domain models (User, Event, EventBooking, Tour, TourBooking, Donation, Contact, Favourite, Ticket) to ensure data integrity and business logic correctness. I developed and tested three key user-facing features: the Visit Us page, Support page, and Donation page, including their associated controllers. I performed bug fixes and UI refinements across multiple pages to improve consistency and user experience. I created and maintained the OpeningHours database table and seeding logic in DbSeeder.cs. Additionally, I authored 45+ unit test cases covering model validation, data constraints, relationships, and business logic

algorithms, achieving a 100% test pass rate. **Technical Limitations:** A challenge encountered was maintaining consistency between client-side and server-side validation. While client-side validation improves user experience through immediate feedback, server-side validation must be independently implemented for security. Future work would benefit from centralising validation rules in a dedicated service layer to reduce duplication and establish a single authoritative validation model.

Kai Wallace-Sahel – M01012141

As a developer it was my role to build the accounts section of the website, I was given the task to create the functional Login page, View tickets page, View saved page and Accounts page I created these using cshtml with controller logic that coincide with their respective pages. The controllers Account and Login linking the site to the database so the data can be displayed dynamically. These controllers have action methods to handle user requests and return the correct data. When a user enters their login detail the system checks the user ID against the user table in the database and if the details match the user is directed to their account where they can view their favoured information on the account.

One of the main issues I faced was resolving version control conflict when merging work into the master branch because multiple members were editing shared files, conflict occurred that required me to have to review the changes in git and combine them carefully without breaking anything. I also experienced issues where updates that were made were not appearing on the website, this was resolved by rebuilding my files on the master, clearing cache and restarting Visual Studio.

Reflecting on past work if I was to do this again, I would spend more time improving on how changes were shared, some of the problems could have been reduced by making sure the shared files were updated more carefully.

Tanzira Khan Khanam – M00996376

This project taught me a lot about adapting when things don't go as planned. I started with technical issues in Visual Studio, which stopped me from working on my pages, so I switched to Visual Studio Code to keep up. Building the HTML for the Amenities, Privacy Policy, Sign Up, and Membership pages helped me understand the structure of the site before moving back into Razor once my laptop was fixed. Working with both the front end and Razor files made me more confident in how ASP.NET Core blends C# logic with the visual layout.

My main task was developing the donations feature, which pushed me to understand backend development more deeply. I worked on the models, controllers, and database setup, and although I struggled at first with migrations and connecting everything to SQL Azure, teamwork helped me get through it. Creating the MockPaymentService also gave me hands-on experience with asynchronous logic and simulating real payment behaviour. Overall, the project helped me grow technically and showed me how important problem-solving and collaboration are in development.

References

Microsoft (2026) *Azure SQL documentation*. Available at: <https://learn.microsoft.com/en-gb/azure/azure-sql/?view=azuresql> (Accessed: 19 April 2026).

Microsoft (2026) *ASP.NET documentation*. Available at: <https://learn.microsoft.com/en-us/aspnet/core/?view=aspnetcore-10.0> (Accessed: 19 April 2026).

Microsoft (2026) *Entity Framework documentation hub*. Available at: <https://learn.microsoft.com/en-us/ef/> (Accessed: 19 April 2026).